

Gas Optimization Report

Project: MultiSigWallet

Course: CS 218 Programmable and Interoperable Blockchains

Instructor: Dr. Subhra Mazumdar

Team: 412

1. Executive Summary

This report outlines the gas optimization strategies implemented in the `MultiSigWallet` smart contract. All optimizations were verified via our Foundry test suite, achieving 100% test coverage across lines, statements, branches, and functions, with 30 tests passing successfully.

2. Gas Usage Comparison

By analyzing the execution costs before and after applying targeted optimizations, we observed significant reductions in both deployment costs and average transaction execution costs.

Metric / Function	Before Optimization	After Optimization	Savings
Deployment Cost	2,152,896	1,940,352	-212,544
Deployment Size	10,372 bytes	9,362 bytes	-1,010 bytes
<code>submitTransaction</code> (Avg)	128,430	120,178	-8,252
<code>approveTransaction</code> (Avg)	70,804	63,043	-7,761
<code>executeTransaction</code> (Avg)	75,863	62,456	-13,407
<code>revokeApproval</code> (Avg)	37,852	30,708	-7,144

Table 1: Comparison of Gas Costs (in gas units)

3. Implemented Gas Optimizations & Reasoning

Optimization		Where Applied	Reasoning
<code>external public</code>	over	All externally-called functions	Avoids ABI re-encoding for internal calls, reducing execution overhead.
<code>calldata memory</code>	over	<code>submitTransaction</code> params	Avoids copying arguments (<code>_data</code> , <code>_description</code>) from <code>calldata</code> to memory.
<code>unchecked { ++i; }</code>		Constructor owner loop	Bounded by <code>ownerCount</code> , making it safe to skip overflow checks and save gas on every iteration.
<code>unchecked mutation</code>	state	<code>approveTransaction</code> & <code>revokeApproval</code>	Bounded by number of owners or guarded by a <code>TxNotApproved</code> check, bypassing safe math checks.
Cached array length		<code>uint256 ownerCount = _owners.length</code>	Saves repeated expensive <code>SLOAD</code> operations in loop conditions.
Cached storage read		<code>executeTransaction</code>	Saves one <code>SLOAD</code> by caching <code>requiredApprovals</code> in memory.
storage pointer		Transaction storage <code>txn</code>	Prevents expensive structural copies to memory when loading a transaction.
$O(1)$ owner check		<code>mapping(address => bool) isOwner</code>	Avoids an $O(N)$ loop over the <code>owners[]</code> array when verifying caller permissions.

Table 2: Summary of Optimization Strategies